



AT ALL NETWORKS

Inheritance & Polymorphism

Program



AT ALL NETWORKS

Classes, objects & Encapsulation

Inheritance & Polymorphism

Composition, Interfaces & special methods

SOLID principle & clean code

Design Patterns

Testing & Project structure





AT ALL NETWORKS

Inheritance

Inheritance



AT ALL NETWORKS



```
class Book:
    def __init__(self, title):
        self.title = title
        self._available = True

    def borrow(self):
        if not self._available:
            raise ValueError("Not available")

        self._available = False
```

```
class DVD:
    def __init__(self, title):
        self.title = title
        self._available = True

    def borrow(self):
        if not self._available:
            raise ValueError("Not available")

        self._available = False
```



```
class LibraryItem:
    def __init__(self, title):
        self.title = title
        self._available = True

    def borrow(self):
        if not self._available:
            raise ValueError("Not available")

        self._available = False
```

```
class Book(LibraryItem):
    pass

class DVD(LibraryItem):
    pass

class Magazine(LibraryItem):
    pass
```

Benefits of inheritance



AT ALL NETWORKS

Code reuse

Shared logic lives in one place; subclasses get it for free without copy-paste

Single point of fix

A bug corrected in the parent propagates to all subclasses automatically

Extensibility

You can add new types without touching existing code (this is what OCP is built on)

Uniform interface

All subclasses share the same method signatures, so calling code doesn't need to know which concrete type it's dealing with

Logical hierarchy

The code mirrors how we naturally classify things ("a Book *is a* LibraryItem"), making it easier to reason about

Polymorphism

One function can work on any subclass transparently, as long as it respects the parent's interface

Inheritance



AT ALL NETWORKS

Single inheritance

```
class Animal:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def describe(self) -> str:
        return f"{self.name}, {self.age} years old"

    def speak(self) -> str:
        return "..."
```

Inheritance allows a "**child**" class, to inherit properties and method from a "**parent**" class.

It's used to **share code** between classes.

In this example :

- **Animal** is the **parent** class
- **Dog** is the **child** class

```
class Dog(Animal):
    def __init__(self, name: str, age: int, breed: str):
        super().__init__(name: name, age: age)
        self.breed = breed

    def speak(self) -> str:
        return "Woof!"

    def describe(self) -> str:
        base = super().describe()
        return f"{base}, breed: {self.breed}"
```

in the `__init__()`, **super()** is used to **delegate shared setup**

speak() method is fully **override**

here, **super()** is used to **extend**

Inheritance

Method overriding

Method **overriding** is when a subclass defines a method with **the same name** as one in its parent, replacing or modifying its behaviour for that specific type.



*Python does not support **Overloading** (because of dynamic typing) though similar behavior can be achieved using default or variable arguments.*



AT ALL NETWORKS

Full override

- the child **replaces** the parent's behaviour entirely. The parent's version is irrelevant for this type, so **no super() is needed**.

Extension

- the child wants to keep the parent's behaviour and **add to it**. **super() is called** to get the parent's result, then the child builds on top.

No override

- the child simply inherits the **parent's method as-is**, without redefining it. This is the default when the parent's behaviour fits the subclass perfectly.

Inheritance



AT ALL NETWORKS

`super()`

`super()` is a way to call the parent class's constructor and other **methods**.

Situation	Use <code>super()</code> ?	Why
<code>__init__</code> in a subclass	Always	Parent must set up its own attributes – skip it and they don't exist
Overriding a method, keeping parent logic (extend)	Yes	Call <code>super().method()</code> first (or last), then add your own behaviour
Overriding a method, replacing it entirely (fully override)	No	You don't need the parent's version at all – like <code>Dog.speak()</code>
<code>__str__</code> / <code>__repr__</code>	Usually yes	Build on the parent's string rather than rewriting it from scratch
<code>__eq__</code> / <code>__hash__</code>	Rarely	These are usually fully redefined based on the subclass's own fields
Multiple inheritance	Always	Critical – <code>super()</code> follows MRO order and chains all <code>__init__</code> calls correctly; hardcoding the parent's name breaks the chain

Inheritance



AT ALL NETWORKS

Multiple inheritance

Multiple inheritance is when a class inherits from **more than one parent** simultaneously, gaining the attributes and methods of all of them. One parent is the **real "is-a" relationship**, the others are **capability add-ons**. When you have two genuine "is-a" parents, the design usually has a problem.

Method Resolution Order

The **MRO** uses the **C3 linearization algorithm**, which produces a **consistent, predictable order**. Python walks this list **left to right** and uses the **first class that defines the method**.

A class always comes before its parents

The order you write the parents in is respected – class D(B, C) puts B before C

No class appears twice in the chain

```
class A:
    def hello(self): return "A"

class B(A):
    def hello(self): return "B"

class C(A):
    def hello(self): return "C"

class D(B, C):
    pass
```

```
print(D.__mro__)
```

```
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,
<class '__main__.A'>, <class 'object'>)
```

Inheritance



AT ALL NETWORKS

Abstract classes

An **abstract class** is a class that is intentionally **incomplete**. It defines a **structure** – some methods, possibly some concrete behaviour – but leaves certain methods **unimplemented**, marked with **@abstractmethod**. It exists to be subclassed, **never instantiated directly**.

*"Would it ever make sense to **instantiate the parent directly**?"*

- **No** → make it **abstract**
- **Yes** → keep it **concrete**

```
from abc import ABC, abstractmethod

class LibraryItem(ABC):
    def borrow(self):
        self._available = False

    @abstractmethod
    def borrow_duration_days(self) -> int:
        ...
```

```
item = LibraryItem() # TypeError: Can't instantiate abstract class
```



If a **subclass** does **not implement all abstract methods or properties** from its parent, it also **remains abstract** and cannot be instantiated.



Abstract classes can **have concrete methods alongside abstract ones** – this is actually one of their **strengths** over **Protocol**, since you can provide default behaviour while still enforcing that certain methods are overridden

Inheritance



AT ALL NETWORKS

Abstract classes

An **abstract class** is meant to :

Prevents direct instantiation

- you can't accidentally create a "half-built" object

Enforces a contract on subclasses

- if a subclass doesn't implement every abstract method, Python refuses to instantiate it too

Some **abstract class** use cases :

shared behaviour + enforced contract

- This is the classic inheritance use. The abstract class provides some concrete methods (reused by all subclasses) and marks others as abstract (each subclass implements its own version).

pure interface, no shared behaviour

- Sometimes an abstract class has *only* abstract methods and no concrete code at all. In this case it's functioning purely as an interface — defining what a type must be able to do, with no implementation provided.

	ABC	Protocol
Requires inheritance	Yes	No
Can carry concrete methods	Yes	No (usually)
Checked at	Instantiation	Static analysis (mypy)
Best for	Shared behaviour + contract	Structural contract only

Inheritance



AT ALL NETWORKS

*"If it walks like a duck, swims like a duck
and quacks like a duck, it's a duck."*



Duck typing

Python doesn't check what an object **is** — it checks what it **can do**. As long as an object has the right methods, Python will use it, regardless of where it sits in any class hierarchy.

Use cases

- Functions that operate on a **behaviour**, not a type
- Collections of mixed types
- Plugin and extension systems

```
def export(item) -> str:  
    # works on any object with to_csv()  
    return item.to_csv()
```

```
for item in items:  
    # each responds in its own way  
    print(item.describe())
```

	Inheritance	Duck typing
Contract enforced by	Class hierarchy	Method presence at runtime
Error appears	At instantiation (with ABC)	At runtime, when method is called
Requires shared parent	Yes	No
Best for	Closely related types	Loosely coupled, flexible code

Inheritance



AT ALL NETWORKS

Liskov Substitution Principle (LSP)

A **subclass** must be **usable anywhere its parent is expected**, without the calling code needing to know the difference.

LSP is used in **SOLID** principles because it describes a fundamental property of **well-designed inheritance**.

Three things that **violate** LSP :

Throwing an exception
where the parent doesn't

Weakening a guarantee the
parent made (returning
None when the parent
always returns a value)

Requiring more from the
caller than the parent does
(stricter preconditions)

Example :

If we have Magazine as child of LibraryItem with a method that allows to borrow, Magazine should be borrowable too.

```
def borrow(self) -> date:
    # This is a violation of Liskov Separation Principle
    # Throwing an Exception where the parent doesn't
    raise RuntimeError("Magazines are for in-library use only.")
```

Inheritance



AT ALL NETWORKS

Liskov Substitution Principle (LSP)



```
class LibraryItem(ABC):
    def __init__(self, title):
        self.title = title
        self._available = True

    def borrow(self):
        # promise: works on
        if not self._available: # available items
            raise ValueError("Not available")
        self._available = False

class Book(LibraryItem): pass # fine
class DVD(LibraryItem): pass # fine
```

```
class Magazine(LibraryItem):
    def borrow(self):
        #Break the promises
        raise RuntimeError("Magazines cannot be borrowed")

# Calling code must defend itself
def checkout(item: LibraryItem):
    if isinstance(item, Magazine): # LSP smell
        return
    item.borrow() # still unsafe
```



```
class LibraryItem(ABC): # common root only
    def __init__(self, title):
        self.title = title
        # no borrow() here – no false promise

class BorrowableItem(LibraryItem):
    def borrow(self):
        # promise lives here
        if not self._available:
            raise ValueError("Not available")
        self._available = False

class ReferenceItem(LibraryItem):
    pass # no borrow() – honest
```

```
class Book(BorrowableItem): pass
class DVD(BorrowableItem): pass
class Magazine(ReferenceItem): pass

# Calling code is clean – no isinstance needed
def checkout(item: BorrowableItem):
    item.borrow() # always safe
```

Inheritance



AT ALL NETWORKS

Mixin patterns

A Mixin is a class that packages a **single, reusable capability** and is designed to **be combined into another class via multiple inheritance** — **never instantiated** on its own. It contributes working code (real methods, real state), but it does **not** represent a standalone concept.

Each Mixin does exactly one thing.

They don't know about each other, don't depend on each other, and can be combined in any order.

super() is non-negotiable Mix and Match freely

Adding a new capability means writing one new Mixin and adding it to the class declaration — nothing else changes.

Mix and match freely

Adding a new capability means writing one new Mixin and adding it to the class declaration — nothing else changes.

Use cases

Serialisation

- Adds `to_json()` / `to_dict()` to any model class. Used heavily in Django REST Framework.

Logging

- Adds structured logging to any service class without wiring up a logger in every class individually.

Authentication

- Django's `LoginRequiredMixin` adds auth checking to any view with a single line of inheritance.

Inheritance



AT ALL NETWORKS

Example of differences with interfaces from Java

	Java interface	Python Mixin	Python ABC	Python Protocol
Carries implementation	No	Yes	Optional	No
Carries state	No	Yes	Optional	No
Requires inheritance	Yes	Yes	Yes	No
Prevents instantiation	Yes	No	Yes	No
Closest to...	Contract	Reusable behaviour	Enforced contract	Structural contract

Inheritance



AT ALL NETWORKS

MECHANISM	WHAT IT IS	REQUIRES INHERITANCE	CARRIES IMPLEMENTATION	CONTRACT ENFORCED	BEST USED FOR
Inheritance	A subclass derives attributes and methods from a parent class	Yes	Yes	Optional (with ABC)	Genuine "is-a" relationships with shared behaviour
ABC	Abstract base class — defines a contract, optionally with shared concrete methods	Yes	Optional	Yes — at instantiation	Enforcing that subclasses implement required methods
Mixin	A small class that adds one specific capability, designed to be combined via multiple inheritance	Yes	Yes	No	Adding reusable behaviour without a deep hierarchy
Protocol	A structural interface — defines required methods without forcing inheritance	No	No	Static analysis only (mypy)	Describing what an object must do, not what it must be
Duck typing	No contract at all — Python calls the method and raises at runtime if it's missing	No	No	No — runtime only	Flexible, loosely coupled code between unrelated types
Multiple inheritance	A class inherits from more than one parent; Python resolves conflicts via MRO	Yes	Yes	No	Combining a real parent with one or more Mixins

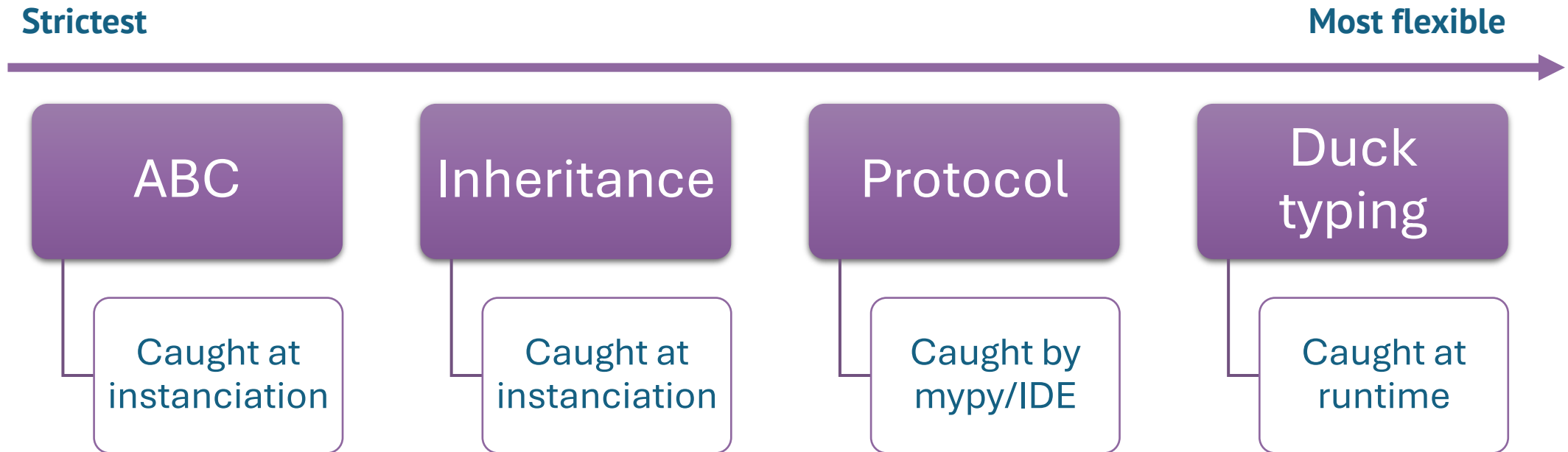
Polymorphism



AT ALL NETWORKS

Polymorphism

Polymorphism is the ability to call the **same method** on **different objects** and get the right **behaviour for each**, without the calling code knowing or caring which type it's dealing with.



Polymorphism is the **goal**. Everything else is the **tool** you choose depending on **how much safety and structure** you want around that goal. The question is never "should I use polymorphism?" – you almost always want it. The question is "**which mechanism gives me the right level of enforcement for this situation?**"



AT ALL NETWORKS

We know the way!